

ClientPilot

Build Plan & System Architecture — Version 1

Owner: Hardik | Budget: Rs 0 | Goal: Rs 20,000–30,000/month from freelance work
Decisions locked: Gmail manual-send outreach | Four discovery sources | Deterministic core, AI optional

0. What this document changes from the original concept

The original ClientPilot spec is good on vision and weak on the three things that decide whether it earns money. This plan keeps the vision and fixes those three.

Area	Original spec	This plan
Sending	Resend / Gmail API, "30/day automated"	Gmail, you press send . ClientPilot drafts and queues; it never sends. Zero deliverability risk, zero account bans, legal by default.
Discovery	GitHub-first	Four sources ranked by buying intent . Job boards first (hiring = budget), then Product Hunt, then directories, then GitHub.
Hosting	Cloudflare Workers for everything	Workers can't crawl and audit sites (CPU + subrequest limits). Split: GitHub Actions runs the crawler, Vercel/Workers serves the dashboard.
AI	Optional layer	Same — and V1 ships with it off . Templates + evidence slots beat generic AI prose for cold outreach anyway.
Scope	14 tables, 8 engines	5 tables, 4 engines, running end-to-end in ~3 weeks.

The single most important design decision. ClientPilot is a **research and drafting machine**, not a sending machine. The moment it sends email autonomously you inherit spam law, Gmail's account limits, and a reputation you can't repair. Keeping a human on the send button costs you 10 minutes a day and removes the entire category of risk. At 20–30 sends/day that is not a bottleneck — your bottleneck is finding leads worth writing to, and that is exactly what the machine does for you.

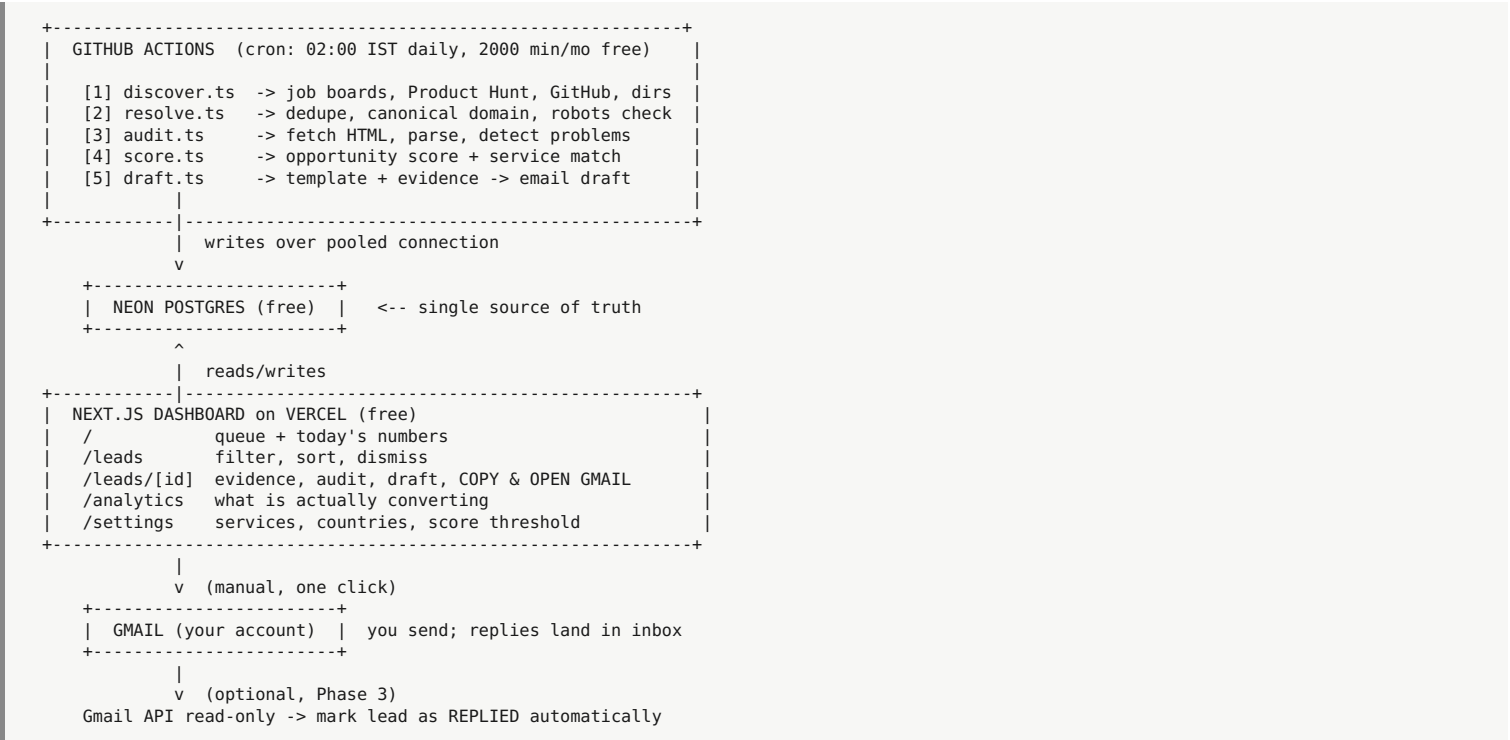
1. The loop, honestly stated

ClientPilot does this (automatic, overnight)	You do this (10 min/day)
1. Discover companies from 4 sources	
2. Resolve + fetch their website	
3. Audit it -> find concrete problems	
4. Score the opportunity 0-100	
5. Pick the service to pitch	
6. Draft a personalised email	
7. Queue it in the dashboard	8. Read draft, edit 1 line, SEND
	9. Reply arrives -> you reply
10. Track sends, detect replies	
11. Remind you about follow-ups	12. You send follow-up

Realistic arithmetic at 20 sends/day, 5 days/week: ~400 sends/month → 6–12% reply rate on evidence-based cold email → ~30 replies → ~6 real conversations → 1–2 projects. At Rs 15k–25k per project that is your target. **The system does not need to be big. It needs to be consistent.**

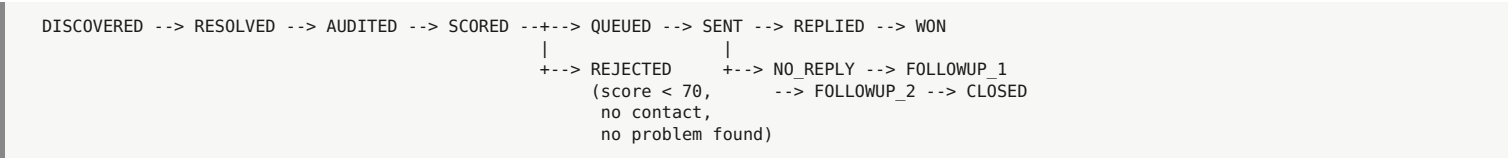
2. System architecture

2.1 Physical topology (all free tiers)



Why GitHub Actions instead of Cloudflare Cron. Auditing one website means 1 DNS lookup + 1-3 HTTP fetches + HTML parsing. A Worker is capped on CPU time and subrequests per invocation, and cannot run a headless browser. A GitHub Actions job is a full Ubuntu VM with Node, 2000 free minutes a month, native cron, and logs. Auditing 100 sites takes ~4 minutes. This is the right tool and it costs nothing.

2.2 Data flow and state machine



Every lead sits in exactly one state. Each pipeline stage only reads leads in the previous state, so the whole system is restartable and idempotent — if a nightly run dies halfway, tomorrow's run picks up exactly where it stopped. This matters more than it sounds: free infrastructure fails regularly.

3. Discovery engine — the four sources, ranked

Ranked by **buying intent**, which is the only ranking that matters. A company that is hiring a frontend developer has an approved budget for frontend work. A GitHub org has code.

#	Source	How to access it free	What you get	Intent signal
1	Job boards Remoteive, Remote OK, WeWorkRemotely, Himalayas	Public JSON/RSS feeds, no key. Filter roles: frontend, React, Next.js, webflow, landing page, web designer.	Company name, site, role, location, posted date	Highest. Budget exists and is allocated right now. Angle: "you're hiring for X — I can do the first project on contract while you search."
2	Product Hunt / new launches	Public GraphQL API (free dev token) or the daily RSS. Take launches 7-45 days old.	Product, tagline, site, maker, socials	High. Just launched, cares about conversion, usually pre-team, founder reachable and responsive.
3	Niche + local directories Indie Hackers, BetaList, SaaS directories, local Jaipur/India business listings	Polite scraping of public listing pages, robots.txt respected, 1 req/sec.	Company, site, category, sometimes email	Medium. Broad volume. Local Indian businesses convert at lower ticket but far faster — good for your first 2 clients.
4	GitHub orgs	REST API. Always authenticate — 60 req/hr unauth vs 5,000 authenticated with a free PAT.	Org, homepage URL, repo stack, contact	Lowest. Developers buy from developers least often. Use only as a stack-detection enricher for leads found elsewhere.

Practical target mix per nightly run: 40 from job boards, 25 from Product Hunt, 25 from directories, 10 GitHub-enriched. ~100 raw leads/night → roughly 25 survive auditing and scoring → more than enough to keep a 20/day send queue full.

Scraping rules that keep you out of trouble

- Honour `robots.txt` — check it before fetching, cache the result per domain.
- One request per second per domain; global concurrency 5.
- Honest User-Agent identifying you and your email.
- Public pages only. Never log in, never bypass a paywall, never touch a CAPTCHA.
- Store only business contact data (company email, public founder email). No personal-address harvesting.

4. Website analyser — the part that creates your leverage

This is what makes your email not-spam. You are not saying "do you need a website." You are saying "your H1 says nothing and your CTA is below the fold on mobile — here's a fix." **Evidence is the product.**

4.1 What gets fetched

Plain `fetch()` of the homepage with a 10s timeout, plus `/pricing` and `/about` if linked. Parse with **cheerio** (fast, no browser). Optionally one Lighthouse run via `@lhci/cli` inside the Action for the top-scoring 20 leads only — it is slow, so it is not run on everything.

4.2 Detection rules (all deterministic, all explainable)

Check	Rule	Problem tag	Weight
Missing viewport meta	No <code><meta name="viewport"></code>	NOT_MOBILE_READY	9
Weak H1	Missing, empty, <15 chars, or equals company name only	UNCLEAR_VALUE_PROP	8
No CTA above fold	No button/link in first 2 sections matching cta verbs (get, start, book, try, demo, contact)	NO_CLEAR_CTA	9
Missing meta description	Absent or >160 / <50 chars	WEAK_SEO	4
Broken internal links	HEAD request on nav links returns 4xx/5xx	BROKEN_LINKS	7
Heavy page	HTML + inline assets > 2 MB, or >25 render-blocking scripts	SLOW_SITE	7
Legacy stack	jQuery-only, table layouts, no responsive CSS, old builder signatures	OUTDATED_BUILD	8
No social proof	No testimonial/logo/review keywords anywhere on homepage	NO_TRUST_SIGNALS	5
No HTTPS / bad cert	http:// only or TLS error	INSECURE	6
No analytics	No GA/Plausible/PostHog tag detected	NO_MEASUREMENT	3

Discipline rule: a lead with **zero** detected problems is **rejected**, no matter how attractive the company looks. If you can't point at something concrete, the email becomes generic, and generic email is spam. This one rule protects the whole system's reply rate.

4.3 Contact discovery

In priority order: `mailto:` on the site → `/contact` page → job posting contact → Product Hunt maker's public profile → GitHub public profile email. **No pattern-guessing** (`firstname@domain`) — guessed addresses bounce, and bounces are what kill a Gmail account. No contact found = lead is parked, not sent.

5. Opportunity scorer

SCORE (0-100)		
Problem severity	0-30	sum of matched rule weights, capped at 30
Service fit	0-25	do the problems map to services you enabled?
Buying intent	0-20	job board 20 PH launch 15 directory 8 github 4
Contact quality	0-15	direct founder email 15 role email 10 form only 3
Evidence strength	0-10	can you name a specific visible thing? 10 : 4
>= 80	ACT NOW	-> top of the send queue
70-79	GOOD	-> queue behind the 80s
< 70	REJECT	-> archived, never shown to you

Threshold is configurable in settings, but start at **70** and raise it — never lower it. If the queue runs dry, fix discovery, don't lower the bar.

6. Outreach engine (drafts only)

6.1 Structure of every email

Subject: quick note on {company}'s homepage		(lowercase, <45 chars, no hype)
Hi {first_name},		
{SOURCE HOOK}	one line proving you actually looked e.g. "Saw you're hiring a frontend dev at Acme."	
{EVIDENCE}	one specific, verifiable observation e.g. "Your hero says 'Acme' but never says what it does, and on mobile the signup button sits below three folds."	
{IMPLICATION}	one line, understated, no fear-mongering	
{OFFER}	small, concrete, low-commitment e.g. "I rewrote the hero section as a quick mockup - want me to send it?"	
{SIGN-OFF}	name, one link, nothing else	
- Hardik		
{portfolio link}		

Under 120 words. No attachments. No images. No tracking pixel (pixels get you flagged and tell you nothing useful at this volume). One link maximum.

6.2 The send flow in the dashboard

```
/leads/[id]
+-----+
| ACME AI                                     score 88  ACT |
| acme.ai - USA - SaaS - found via Remotive (hiring: React dev) |
+-----+
| EVIDENCE                                | DRAFT |
| - H1 is just "Acme"                    | [editable textarea] |
| - No CTA above fold (mob.)              | |
| - 3.1 MB homepage                      | [ Copy draft ] |
| - No testimonials                      | [ Open in Gmail ] -> compose |
|                                         | prefilled via mailto: |
| CONTACT alex@acme.ai                   | [ Mark as sent ] [ Dismiss ] |
+-----+
```

"Open in Gmail" builds a `https://mail.google.com/mail/?view=cm&to=...&su=...&body=...` URL. No API, no OAuth, no quota, nothing to break.
"Mark as sent" stamps the timestamp and schedules follow-ups at day 4 and day 9.

7. Follow-ups and reply handling

V1 is a **reminder system**, not an automation. Day 4 and day 9 the lead reappears in the dashboard queue with a pre-written two-line follow-up. Two follow-ups maximum, then the lead is CLOSED permanently. You mark replies manually in one click — it takes two seconds and you were reading the inbox anyway.
Phase 3 upgrade: Gmail API with **read-only** scope. It scans for messages from known lead addresses, auto-marks REPLIED, and cancels pending follow-ups. Read-only means no sending permission and therefore no risk profile change.

8. Database schema (5 tables, V1)

leads		
id	uuid pk	
company	text	
domain	text unique	-- canonical, dedupe key
country	text	
industry	text	
source	text	-- job_board product_hunt directory github
source_meta	jsonb	-- role hiring for, PH votes, etc
status	text	-- state machine value
score	int	
created_at	timestampz	
updated_at	timestampz	
audits		
id	uuid pk	
lead_id	uuid fk -> leads	
fetched_at	timestampz	
http_status	int	
title	text	
h1	text	
meta_desc	text	
tech	text[]	-- detected stack
problems	jsonb	-- [{tag, weight, evidence_string}]
page_bytes	int	
raw_snapshot	text	-- truncated HTML, for debugging
contacts		
id	uuid pk	
lead_id	uuid fk -> leads	
name	text	
role	text	
email	text	
confidence	text	-- direct role form_only
source_url	text	
outreach		
id	uuid pk	
lead_id	uuid fk -> leads	
kind	text	-- initial followup_1 followup_2
subject	text	
body	text	
service	text	-- which service was pitched
drafted_at	timestampz	
sent_at	timestampz null	-- null = still in queue
replied_at	timestampz null	
reply_class	text null	-- hot interested maybe not_now no auto
settings		
id	int pk default 1	-- single row
services	text[]	
countries	text[]	
min_score	int default 70	
daily_limit	int default 20	

Indexes: `leads(status, score desc)`, `leads(domain)`, `outreach(lead_id)`, `outreach(sent_at)` where `replied_at` is `null`. That is the entire performance story at this scale.

9. Tech stack, final

Layer	Choice	Why this and not the alternative
Dashboard	Next.js 15 (App Router) + Tailwind, on Vercel	You already know it. Server Components read Postgres directly — no separate API layer to build.
Database	Neon Postgres, free tier	Serverless driver over HTTP survives Vercel's connection model. Supabase is the fallback. Both sleep when idle; the pipeline tolerates cold starts.
ORM	Drizzle	Lightweight, SQL-shaped, works in both Next.js and the Actions script. Prisma's engine is heavy for a free-tier setup.
Pipeline runtime	TypeScript scripts run by GitHub Actions cron	Free, logged, restartable, full Node. Same language as the dashboard, shared types.
HTML parsing	cheerio	No browser needed for 95% of checks. Playwright only if you later want screenshots.
Sending	Gmail, manual, via prefilled compose URL	Zero infrastructure, zero risk, zero cost. Revisit only after you have paying clients.
AI	None in V1. Interface stubbed.	Ship without it. Add Cloudflare Workers AI (free neuron allocation) or Groq's free tier later, behind a flag, for reply classification first — that is where AI actually helps.
Secrets	GitHub Actions secrets + Vercel env vars	Only two needed in V1: <code>DATABASE_URL</code> , <code>GITHUB_PAT</code> .

10. Build plan — 4 weeks, in dependency order

Rule for the whole build: **every week must end with something that runs end to end.** Never build two engines before testing one.

Week 1 — Skeleton and one real audit

- Day 1:** Next.js app, Tailwind, Neon database created, Drizzle schema pushed. Deploy the empty app to Vercel. *Done when: a live URL shows "ClientPilot" and can read a row you inserted by hand.*
- Day 2-3:** Write `audit.ts` as a local CLI: give it a URL, it fetches, parses with cheerio, runs all 10 detection rules, prints a JSON problem list. **Run it against 20 real websites you pick manually.** Tune the rules until the output matches what you'd say if you looked at the site yourself. *This is the most important day of the build.*
- Day 4:** Write `score.ts`. Feed it the 20 audits. Check the ranking matches your instinct.
- Day 5:** Persist audits + scores to Postgres. Build `/leads` as a plain table.

Week 2 — Discovery

- Day 6-7:** Job board adapter (Remotive + Remote OK JSON feeds). Normalise to a common `RawLead` shape. Domain canonicalisation + dedupe against existing rows.
- Day 8:** Product Hunt adapter.
- Day 9:** Directory scraper with robots.txt check and rate limiting. GitHub enricher with an authenticated PAT.
- Day 10:** Contact extraction. Wire discovery → audit → score into one `pipeline.ts`. Run it locally end to end. *Done when: you run one command and 25 scored leads with contacts appear in the dashboard.*

Week 3 — Outreach and going live

- Day 11-12:** Template engine: map problem tags → service → template variant. Write 4 templates by hand (hiring hook, new-launch hook, redesign hook, local-business hook). Generate drafts into the `outreach` table.
- Day 13:** Lead detail page: evidence panel, editable draft, Copy, Open in Gmail, Mark as sent, Dismiss.
- Day 14:** Move `pipeline.ts` into a GitHub Actions cron workflow. Confirm the nightly run works unattended.
- Day 15: Send your first 10 emails by hand.** Not 30. Ten. Read every draft fully before sending — you are QA-ing the machine, not using it yet.

Week 4 — Tighten based on reality

- Day 16-18:** Follow-up scheduling + queue view. Reply marking + classification buttons. Analytics page (sends, replies, reply rate by source and by template).
- Day 19-20:** Settings page. Then **read your first 40 sent emails and the replies**, and fix the templates and detection rules that produced weak ones. Kill whichever source has the worst reply rate.

After V1 — in strict priority order

- Gmail API **read-only** reply detection (removes the only manual bookkeeping left).
- AI for **reply classification** only — the one place it genuinely beats rules.
- Screenshot capture of the problem area, attached as evidence in the dashboard (huge on reply rates when you mention it).
- AI-assisted draft variation, behind a flag, A/B tested against the hand-written templates. **Only ship it if it wins.**
- More discovery sources — last, not first.

11. Failure modes and the honest risks

Risk	Why it happens	Mitigation built into this plan
Gmail flags your account	Volume, bounces, spam reports	≤20/day, human-reviewed, no guessed emails, real reply-to, easy opt-out line
Zero replies	Generic evidence, weak offer	Reject leads with no detected problem; offer a free deliverable, not a call
Scraper breaks silently	Sites change markup	Actions logs + a "leads discovered last night" counter on the home page; zero means alert
Free tier limits hit	Neon compute hours, Actions minutes	Nightly run is ~5 min/day = 150 min/mo of 2000; pipeline is single-pass and idempotent
You stop using it	Most likely failure by far	10 min/day ritual, queue capped at 20 so it never feels like a backlog
Legal (GDPR/CAN-SPAM)	EU/UK targets, B2B cold email	Business contacts only, identify yourself, state why you're writing, honour opt-outs immediately, keep a suppression list

12. What to do on day one

1. Create: Neon project, Vercel project, GitHub repo, a GitHub PAT with public-repo scope.
2. Write down **five services you will actually deliver well**. Put them in settings. The scorer's "service fit" dimension is meaningless without this.
3. Build the audit CLI and point it at 20 sites you already think are badly built. **Do not touch the dashboard until that output is good.**

The measure of success for V1 is not features. It is: on day 15, ten real, specific, non-embarrassing emails went out to ten real companies, and you didn't write them from scratch. Everything else is optimisation.